

UI Design

The user interface is designed to help users easily navigate through different parts of the game, such as team details, player statistics, league standings, and match results. The focus is on clarity and usability, since this is a manager game rather than an action-heavy one.

The UI layer is completely separate from the core game logic. It only handles what the user sees and interacts with — it does not contain any sport rules or match calculations. Sport-specific rules are accessed through the ISport interface, while team, player, match, and league data are provided by their corresponding system classes.

This separation keeps the UI independent from any specific sport implementation and makes it straightforward to add new sports later.

Screens

Main Menu: This is the first screen the user sees when launching the application. From here, users can start a new game, select the sport they want to manage, or exit. The list of available sports is determined by which classes implement the ISport interface. The design should be simple and direct, giving easy access to the main actions without unnecessary clutter.

Team Screen: This screen shows information about the player's managed team — the team name, logo, and squad list. It may also include basic player status information like injuries or fitness levels. Data on this screen comes from the BaseTeam class and the player entities that extend BasePlayer.

League Table: This screen displays the current standings of all teams in the league, including matches played, wins, losses, draws, and total points. The data is managed by the League class and updates after each match is played.

Match Screen: This screen shows the two competing teams and the current score during a match. It may also display key match events as they happen. Match outcomes are calculated by the MatchEngine, and match data is stored in the Match class.

Player List: This screen lists all players in the team with basic information such as name, position, and relevant attributes. It may also show current fitness or injury status. The data comes from player classes that extend BasePlayer, such as FootballPlayer or HeadballPlayer.

Extensibility

The system is built to support multiple sports without touching the core framework. Every sport must implement the ISport interface, which requires defining three things: the number of players needed per team (`getRequiredPlayers()`), the match duration (`getMatchDuration()`), and the points awarded for a win (`getPointForWin()`). FootballSport and HeadballSport are two existing examples of this.

Since the game logic only communicates with sports through the ISport interface, adding a new sport is as simple as writing a new class that implements it. Nothing else in the system needs to change.

Conclusion

This project builds a flexible sports manager simulation by keeping the user interface, game logic, and sport-specific rules in separate layers. Abstract classes and interfaces make it easy to extend the system — new sports, new player types, or new features can be added without modifying what already exists. The result is a clean, maintainable foundation that can grow as the project develops.